

Cloud Computing Tools

Deliverable 3: Diseño e implementación de flujos de datos automatizados con AWS Lambda

Juan Pablo Jiménez — Maestría en Analítica Aplicada
Harrison Alonso — Maestría en Analítica Aplicada
Juan Fernando Serrano — Maestría en Analítica Aplicada
Universidad de La Sabana
Maestría en Analítica Aplicada
Prof.: Hugo Franco, Ph.D.
1 de septiembre de 2026

Código fuente: <https://github.com/juanferrrr/Deliverables-Grupo-Alonso-Jimenez-Serrano>

1. Orquestación ETL idempotente con AWS Lambda

1.1. Descripción del problema

En el taller de laboratorio se desarrolló una función Lambda como motor ETL efímero, capaz de leer datos de una fuente externa, transformarlos y persistirlos en Amazon RDS mediante sentencias parametrizadas. El riesgo operativo que motiva este entregable es la re-ejecución: una función Lambda puede dispararse más de una vez sobre el mismo conjunto de datos —por un reintento automático de AWS, por una prueba manual, o por una programación periódica futura vía EventBridge— y un simple INSERT fallaría por violación de llave o, peor, duplicaría registros silenciosamente si la tabla no tuviera una restricción de unicidad.

El ejercicio evaluable consistió en adaptar la función de la guía base (que originalmente lee de Google Sheets) para ingerir en su lugar la categoría de datos ya trabajada en los talleres anteriores —restaurantes de Chía obtenidos de la API de Foursquare—, leyendo el CSV crudo directamente desde el Data Lake en S3 (raw_zone/restaurants_raw.csv) y persistiéndolo en la tabla datos_externos creada en RDS, con la garantía de que ninguna re-ejecución genere registros duplicados.

1.2. Método de solución

La función Lambda (Python 3.12) combina dos librerías con responsabilidades disjuntas: boto3, el SDK de AWS, se usa exclusivamente para leer el objeto CSV desde S3; pg8000, una librería de PostgreSQL escrita enteramente en Python (sin extensiones compiladas en C, a diferencia de psycopg2), se usa para la conexión y las sentencias contra RDS —evitando así incompatibilidades entre el sistema operativo de desarrollo y el runtime Linux de Lambda. Las dependencias se empaquetaron en AWS CloudShell con `pip install --platform manylinux2014_x86_64 --only-binary=:all: --python-version 3.12 -t .`, replicando el mismo comando de la guía base para garantizar compatibilidad binaria con el runtime de Lambda.

Cada fila del CSV se transforma antes de cargarse: el campo categories —serializado en la fuente como el `str()` de una lista de diccionarios de Python (comillas simples, no JSON válido)— se reconstruye con `ast.literal_eval()` y se re-serializa con `json.dumps()` para poder castearlo a JSONB en la sentencia SQL; los campos numéricos (latitude, longitude, distance) se convierten explícitamente con `float()/int()` para evitar que lleguen como texto. La idempotencia se implementa con el patrón UPSERT: una sentencia INSERT con una cláusula `ON CONFLICT (name) DO UPDATE`, apoyada en una restricción UNIQUE sobre la columna name de la tabla destino. Adicionalmente, se usó el truco de PostgreSQL `RETURNING (xmax = 0) AS inserted` para que cada fila reporte si la operación resultante fue una inserción real o una actualización, lo cual permite evidenciar la idempotencia sin depender únicamente de un conteo externo.

Algorithm 1. carga_idempotente_s3_a_rds(): booleano

```
intentar
  csv_bytes <- s3_client.get_object(bucket, raw_key)      # boto3
  filas <- csv.DictReader(csv_bytes)
  conexion <- pg8000.native.Connection(host, db, user, pwd)

  para cada fila en filas:
    categorias_json <- json.dumps(ast.literal_eval(fila.categories))
    resultado <- conexion.run(
      "INSERT INTO datos_externos (...) VALUES (...) " +
      "ON CONFLICT (name) DO UPDATE SET ... " +
      "RETURNING (xmax = 0) AS inserted",
      **fila_transformada)
    si resultado.inserted entonces insertados += 1
    si_no actualizados += 1
```

```

    retornar {procesados, insertados, actualizados}
excepto Exception como e
    log_error(tipo(e), mensaje(e))
    retornar {statusCode: 500, error: mensaje(e)}

```

Sentencia SQL de carga parametrizada (UPSERT idempotente):

```

INSERT INTO datos_externos (name, address, categories, latitude, longitude, distance)
VALUES (:name, :address, :categories::jsonb, :latitude, :longitude, :distance)
ON CONFLICT (name)
DO UPDATE SET
    address = EXCLUDED.address,
    categories = EXCLUDED.categories,
    latitude = EXCLUDED.latitude,
    longitude = EXCLUDED.longitude,
    distance = EXCLUDED.distance
RETURNING (xmax = 0) AS inserted;

```

El código está implementado en el archivo adjunto `lambda_function.py`, disponible en el repositorio referenciado en la portada, junto con el paquete de despliegue `lambda_function.zip` (código fuente + dependencia `pg8000` compilada para `manylinux2014_x86_64`).

1.3. Resultados

Para verificar la robustez del flujo se vació la tabla `datos_externos` (`TRUNCATE`) y se ejecutó la función Lambda dos veces consecutivas con el mismo evento de prueba vacío (`{}`), verificando en cada punto el conteo de filas en RDS.

Ejecución	processed	inserted	updated	COUNT(*) resultante
1 — Primera carga	50	50	0	50
2 — Re-ejecución	50	0	50	50

Tabla 1: Resultado de las dos ejecuciones controladas de la función Lambda. Fuente: elaboración propia, a partir del Response y los logs de CloudWatch de cada invocación.

En la primera ejecución, con la tabla vacía, las 50 filas del CSV se insertaron como registros nuevos (`inserted = 50`, `updated = 0`). En la re-ejecución, con el evento idéntico (`{}`), las 50 filas colisionaron contra la restricción `UNIQUE (name)` y la cláusula `ON CONFLICT` las resolvió como actualización (`inserted = 0`, `updated = 50`), sin alterar el número total de registros. El conteo programático en RDS confirma esta estabilidad:

```

SELECT COUNT(*) FROM datos_externos;
-- resultado: 50 (idéntico antes y después de la re-ejecución)

```

Evidencia de la primera ejecución (log de CloudWatch):

Se ha actualizado correctamente la función "deliverable3".

Ejecutando la función: sin errores ([registros](#))

▼ Detalles

Response Format JSON

```
{
  "statusCode": 200,
  "processed": 50,
  "inserted": 0,
  "updated": 50,
  "skipped_sin_nombre": 0
}
```

Resumen

Código SHA-256 VsCjdSMEE0TXCGUFnU3bTIV/20U6aSq6RWlghpmoJFW=	Tiempo de ejecución hace 6 minutos
Versión de la función \$LATEST	ID de solicitud 68dd3c2a-15be-429f-8fdc-e71cdbe0151f
Duración 1613.11 ms	Duración facturada 1614 ms
Recursos configurados 128 MB	Memoria máx. utilizada 99 MB

Resultado de registro

En el área siguiente se muestran los últimos 4 KB del registro de ejecución. [Haga clic aquí](#) para ver el grupo de registros de CloudWatch correspondiente.

```
START RequestId: 68dd3c2a-15be-429f-8fdc-e71cdbe0151f Version: $LATEST
OK: {'statusCode': 200, 'processed': 50, 'inserted': 0, 'updated': 50, 'skipped_sin_nombre': 0}
END RequestId: 68dd3c2a-15be-429f-8fdc-e71cdbe0151f
REPORT RequestId: 68dd3c2a-15be-429f-8fdc-e71cdbe0151f  Duration: 1613.11 ms    Billed Duration: 1614 ms    Memory Size: 128 MB    Max Memory Used: 99 MB
```

Evidencia de la re-ejecución (log de CloudWatch):

Flujos de registros (5) Eliminar Crear flujo de registros Buscar en todas las secuencias de registro

De forma predeterminada, solo se cargan los flujos de registro más recientes.

☐ Coincidencia exacta ☐ Mostrar caducado [Información](#) < 1 > ⚙

☐ Secuencia de registro	☐ Hora del último evento
☐ 2026/09/01/[\$LATEST]f42a6cea117f4998897920d2d69038ac	2026-09-01 03:37:06 (UTC)
☐ 2026/09/01/[\$LATEST]65a0d973f80548e7803f1df24c1e9358	2026-09-01 03:24:00 (UTC)
☐ 2026/09/01/[\$LATEST]c2ad821e93c14eb892364ff606c54f76	2026-09-01 03:11:35 (UTC)
☐ 2026/09/01/[\$LATEST]ca7d998024f9427e8f90ad70cdbecdb5	2026-09-01 03:07:54 (UTC)
☐ 2026/09/01/[\$LATEST]6e930b59da92418b9d172f7cfa5d14a0	2026-09-01 02:59:50 (UTC)

Verificación en RDS del conteo estable de registros:

The screenshot shows the AWS Data console interface. At the top, a SQL query is entered: `SELECT COUNT(*) FROM datos_externos;`. Below the query editor, the results are displayed in a table. The table has a single row with the value 123 in the 'count' column. The table is titled 'datos_externos 1' and has a 'Grid' view selected. The results are shown in a table with 1 column and 1 row.

count
123

2. Discusión y conclusiones

El resultado más relevante de este entregable no es que la carga haya funcionado, sino que el patrón UPSERT demostró ser determinista bajo re-ejecución: la única diferencia observable entre las dos corridas es la distribución interna entre inserted y updated, mientras que el total de filas procesadas y el conteo en RDS permanecen idénticos. Esto valida la premisa de diseño del taller —que una función Lambda debe poder dispararse más de una vez sobre los mismos datos sin degradar la integridad del Data Lake ni de la capa relacional— y deja el flujo listo para automatizarse con una regla de EventBridge sin riesgo de duplicación, extendiendo así el mismo principio de FinOps y confiabilidad aplicado en los talleres anteriores del curso, ahora a nivel de la capa de ingesta serverless.